# Product Requirements Document: Voice-First Rural Health Triage Assistant

*Capstone project, B.Tech IT, VII Semester. Version 1.0*

## 1. Overview

### 1.1 One-line summary

A Hindi voice-in, voice-out assistant that listens to a person describe a health problem, checks for danger signs, retrieves guidance from trusted Indian health guidelines, and replies by voice with an urgency level and simple next steps.

### 1.2 Problem statement

People in rural areas often cannot read English medical content, find typing difficult, and live far from a doctor. They need quick, understandable guidance on one question: how serious is this, and what should I do now? Existing symptom checkers are English-first, text-based and not grounded in Indian public-health guidelines.

### 1.3 Important framing

This is a **triage and first-aid guidance prototype, not a diagnosis tool**. It never names a disease as a diagnosis and never gives drug doses. It is validated against published protocols and expert review, and it has not been clinically tested. This framing must appear in the UI, the report and the viva.

## 2. Goals and non-goals

### 2.1 Goals

- G1: Accept spoken Hindi and return a spoken Hindi answer end to end.
- G2: Classify every case into one of four urgency levels: Emergency, See a doctor today, See a doctor this week, Home care.
- G3: Ground every answer in retrieved, cited guideline text.
- G4: Never miss red-flag emergencies (target: zero under-triage on protocol test cases).
- G5: Produce measurable evidence: WER, triage accuracy, retrieval quality, expert ratings, ablation.

### 2.2 Non-goals

- Diagnosing diseases or prescribing medicines.
- Replacing a doctor or ASHA worker.
- Supporting all Indian languages (Hindi first; Rajasthani/Marwari is a stretch goal).
- Storing identifiable patient records.

## 3. Users and personas

| Persona | Description | Needs |
|---|---|---|
| Rural resident (primary) | Low literacy, Hindi speaker, basic smartphone | Voice interface, simple words, clear action |
| ASHA / ANM worker (secondary) | Field health worker | Quick second reference, cited guideline source |
| Family caregiver | Looking after child or elderly person | Child and pregnancy danger signs |
| Evaluator / examiner | Reviews the project | Evidence, safety design, reproducibility |

## 4. Scope

### 4.1 In scope (MVP)

- Hindi speech to text, and text to speech.
- Symptom extraction and follow-up questions.
- Rule-based red-flag layer.
- RAG over NHM, ICMR and ASHA/WHO material.
- Urgency classification and cited answer.
- Web app with large microphone button.
- Evaluation suite and dashboard.

### 4.2 Stretch

- Rajasthani/Marwari input via Hindi fallback.
- Offline mode with a small local model.
- WhatsApp or IVR (phone call) channel.

- Fine-tuned small model for dialogue format.

## 5. User stories and acceptance criteria

| ID | Story | Acceptance criteria |
|---|---|---|
| US1 | As a villager, I press one button and speak my problem in Hindi. | Audio up to 60 seconds is accepted; transcript is shown. |
| US2 | As a user, I get follow-up questions if details are missing. | System asks at most 3 follow-ups, one at a time, by voice. |
| US3 | As a user with an emergency, I am told to go to hospital immediately. | Red-flag rules fire before any LLM call; response in under 3 seconds; mentions 108. |
| US4 | As a user, I hear the answer in simple Hindi. | Answer is under 80 words, spoken by TTS, text also shown. |
| US5 | As an ASHA worker, I see which guideline the answer came from. | Each answer lists source document name and page. |
| US6 | As an examiner, I can run the evaluation. | One command reproduces all reported metrics. |

## 6. Functional requirements

- FR1: Record audio in browser and upload to backend.
- FR2: Transcribe Hindi speech; show transcript for user confirmation.
- FR3: Extract structured fields: symptom, duration, severity, age group, sex, pregnancy status, danger-sign answers.
- FR4: Run deterministic red-flag rules on extracted fields and raw text.
- FR5: Retrieve top-k guideline chunks using multilingual embeddings.
- FR6: Generate an answer only from retrieved chunks; if retrieval confidence is low, return the safe fallback: "Not sure, please see a doctor."
- FR7: Assign an urgency level; the final level is the more severe of rules output and LLM output.
- FR8: Convert the answer to Hindi speech.
- FR9: Show a persistent disclaimer and emergency numbers.
- FR10: Log anonymous interaction data (no names, no audio stored by default) for evaluation.

# 7. Non-functional requirements

| Area | Requirement |
|---|---|
| Latency | Emergency path under 3 s; normal path under 10 s end to end |
| Safety | Zero under-triage on protocol test set; no diagnosis or dosage output |
| Privacy | No personal identifiers; audio deleted after transcription unless user consents |
| Usability | Usable by a first-time user in under 1 minute; big buttons, icons, voice prompts |
| Reliability | Graceful fallback if ASR, retrieval or LLM fails |
| Portability | Runs in Docker; reproducible with a requirements file and seed |
| Accessibility | Works on low-end Android Chrome; low bandwidth mode |

# 8. System flow, step by step with implementation

## Flow overview

```
User speaks (Hindi)
  -> [1] Audio capture
  -> [2] Speech to text
  -> [3] Transcript confirmation
  -> [4] Symptom extraction (LLM, structured JSON)
  -> [5] Red-flag rule engine  --(emergency)--> [10] Emergency response
  -> [6] Missing info? ask follow-up (loop, max 3) -> back to [2]
  -> [7] RAG retrieval from guidelines
  -> [8] Grounded answer + urgency (LLM)
  -> [9] Safety merge (take the more severe level)
  -> [10] Hindi text to speech + UI card
  -> [11] Anonymous logging and feedback
```

## Step 1: Audio capture

- **What:** Browser records the user's voice.

- **Implementation:** MediaRecorder API, record as WebM/Opus, push-to-talk with a large mic button, visual waveform, 60 s cap.
- **Tech:** React (or plain HTML + JS for MVP), Web Audio API.
- **Output:** audio blob sent via HTTPS POST.

### Step 2: Speech to text (ASR)

- **What:** Convert Hindi audio to text.
- **Implementation:** Convert to 16 kHz mono WAV with ffmpeg; run Whisper (small or medium) with language=hi; compare against AI4Bharat IndicWhisper if time permits.
- **Tech:** faster-whisper, ffmpeg, Python. GPU on free Colab/Kaggle for experiments; CPU int8 for demo.
- **Fallback:** if confidence is low, ask the user to repeat.

### Step 3: Transcript confirmation

- **What:** Show and read back what was understood.
- **Implementation:** Display transcript, TTS reads it back, user taps Yes or speaks again. This catches ASR errors before they affect triage.
- **Tech:** React state machine, TTS from Step 10.

### Step 4: Symptom extraction

- **What:** Turn free speech into structured data.
- **Implementation:** Prompt an LLM to return strict JSON: {symptoms[], duration, severity, age_group, sex, pregnant, danger_signs_mentioned[], missing_fields[]}. Validate with a schema; retry once on invalid JSON.
- **Tech:** LLM via API (for example Gemini, GPT or Claude) or local Gemma/Llama with LoRA; Pydantic for validation; optional IndicTrans2 to translate Hindi to English for English-only components.

### Step 5: Red-flag rule engine (safety layer)

- **What:** Deterministic emergency detection that runs before and independently of the LLM.
- **Implementation:** A YAML file of rules built from WHO IMCI danger signs and NHM/ASHA danger-sign lists (chest pain, breathing difficulty, unconsciousness, seizures, severe bleeding, snakebite, pregnancy bleeding or severe headache with blurred vision, child unable to drink, high fever with stiff neck, and so on). Match on both extracted fields and Hindi/Hinglish keyword lists in the raw transcript.
- **Tech:** Python, YAML, regex plus fuzzy matching (rapidfuzz). Unit-tested with pytest.

- **Output:** if any rule fires, jump straight to Step 10 with the Emergency level.

### Step 6: Follow-up questions

- **What:** Ask for missing critical information.
- **Implementation:** A small decision table of must-know fields per symptom group (for example fever needs duration, age, rash, neck stiffness). Ask one question at a time by voice, maximum 3.
- **Tech:** Python state machine (LangGraph or a simple FSM), session state in Redis or an in-memory dict for MVP.

### Step 7: Retrieval (RAG)

- **What:** Find relevant guideline passages.
- **Implementation:**
  1. Collect PDFs (NHM, ICMR, ASHA modules, WHO IMCI).
  2. Parse with PyMuPDF; clean headers and footers; chunk by heading, about 300 to 500 tokens with overlap.
  3. Attach metadata: document, page, topic, language.
  4. Embed with multilingual-e5 or BGE-M3 so Hindi queries match English text.
  5. Store in FAISS or Chroma; retrieve top 5, optionally re-rank with a cross-encoder or BM25 hybrid.
- **Tech:** PyMuPDF, sentence-transformers, FAISS/Chroma, rank_bm25.
- **Guard:** if top similarity is below a threshold, trigger the safe fallback.

### Step 8: Grounded answer and urgency

- **What:** Produce the response from retrieved text only.
- **Implementation:** System prompt rules: use only the provided passages; no diagnosis; no drug doses; under 80 words; simple Hindi; output JSON {urgency, advice_hi, sources[]}. Include few-shot examples for each urgency level.
- **Tech:** LLM API or the fine-tuned small model; JSON schema validation.
- **Optional fine-tune:** QLoRA on a 1B to 3B model using translated triage dialogues so it reliably follows the format (Unsloth or Hugging Face PEFT on Colab or Kaggle).

### Step 9: Safety merge

- **What:** Final decision on urgency.

- **Implementation:** final_urgency = max(rule_level, llm_level). A post-check scans the output for banned patterns (drug names with doses, "you have <disease>") and replaces unsafe text with the fallback message.
- **Tech:** Python, regex, unit tests.

### Step 10: Response, TTS and UI

- **What:** Speak the answer and show a card.
- **Implementation:** Colour-coded card (red, orange, yellow, green), icon, short Hindi text, source line, "Call 108" button for emergencies, disclaimer. TTS via Piper, AI4Bharat Indic-TTS or gTTS for MVP.
- **Tech:** React + Tailwind (or Streamlit for speed), FastAPI, audio streamed back as MP3.

### Step 11: Logging and feedback

- **What:** Collect evaluation data and user feedback.
- **Implementation:** Store anonymous records (transcript, extracted fields, urgency, sources, latency) and a thumbs up or down. No personal identifiers; audio is not retained by default.
- **Tech:** SQLite or PostgreSQL, SQLAlchemy; a small analytics page.

## 9. Architecture

| Layer | Components | Technology |
|---|---|---|
| Client | Mic UI, answer card, audio player | React, Tailwind, MediaRecorder |
| API | REST endpoints, session handling | FastAPI, Uvicorn, Pydantic |
| Speech | ASR and TTS services | faster-whisper, ffmpeg, Piper/Indic-TTS |
| Reasoning | Extractor, rule engine, answer generator | LLM API or local model, LangGraph/FSM |
| Knowledge | Vector store and metadata | FAISS/Chroma, sentence-transformers |
| Data | Logs, feedback, test cases | SQLite/PostgreSQL |
| DevOps | Packaging and hosting | Docker, GitHub Actions, Hugging Face Spaces / Render / Railway |

### 9.1 API design

| Endpoint | Method | Purpose |
|---|---|---|
| /api/session | POST | Start a session and return session_id |
| /api/transcribe | POST | Audio in, transcript out |
| /api/triage | POST | Transcript or fields in, urgency, advice and sources out |
| /api/tts | POST | Text in, audio out |
| /api/feedback | POST | Thumbs and comment |
| /api/health | GET | Service check |

### 9.2 Database schema (minimal)

- **sessions**(id, created_at, language)
- **turns**(id, session_id, transcript, extracted_json, rule_level, llm_level, final_level, sources_json, latency_ms)
- **feedback**(id, turn_id, rating, comment)
- **test_cases**(id, text, expected_urgency, source_protocol, category)

## 10. Data and knowledge sources

| Purpose | Source |
|---|---|
| Guidelines for RAG | NHM and ICMR publications, ASHA training modules, WHO IMCI charts and similar open WHO material |
| Speech evaluation | Mozilla Common Voice (Hindi), AI4Bharat IndicVoices/Vistaar |
| Medical QA evaluation | MedQuAD, MedMCQA |
| Translation | AI4Bharat IndicTrans2 or Bhashini APIs |
| Triage test vignettes | Cases you build from IMCI, NHM danger signs and published vignette studies; review by a medical student or doctor |

Check each source's licence and cite it in the report.

# 11. Evaluation plan

| Component | Metric | Target |
|---|---|---|
| ASR | Word error rate on health phrases | Report; compare Whisper sizes |
| Red-flag layer | Recall on emergency cases | 100% on protocol set |
| Triage | Accuracy, and separate under-triage and over-triage rates | Under-triage 0% on protocol set |
| Retrieval | Hit rate at 5, MRR | Above 85% on a hand-made question set |
| Faithfulness | Share of claims supported by retrieved text | Above 90% |
| Language quality | Human rating (1 to 5) of clarity and simplicity from 5 to 10 native speakers | Mean 4 or higher |
| Expert safety review | Doctor or medical student rates 50 to 100 outputs as safe, unsafe or unclear | Zero unsafe |
| Latency | p50 and p95 end to end | Under 10 s |

**Ablation (key result):** LLM only, then LLM + RAG, then LLM + RAG + safety rules. Compare unsafe-output rates across the three.

**Honest limits to state:** validated against published protocols and limited expert review; no clinical trial; not for real patient use.

# 12. Testing strategy

- **Unit tests:** rule engine, JSON validation, safety merge (pytest).
- **Integration tests:** audio in to spoken answer out, using recorded samples.
- **Regression set:** the 100 triage test cases run automatically in GitHub Actions on every commit.
- **Robustness tests:** noisy audio, accents, short and rambling speech, Hinglish words, empty input.
- **Usability test:** 5 to 10 real users, record task completion and confusion points.

## 13. Safety, ethics and legal

- Persistent disclaimer: not a doctor, not a diagnosis, call 108 or 112 in emergencies.
- No diagnosis names, no drug dosages, no pregnancy or child medication advice beyond "see a doctor".
- Safe fallback when unsure.
- Anonymous data only; consent screen for any audio storage; delete on request.
- Cite sources and respect document licences.
- Document known failure modes and bias risks (dialect, gender, accent).

## 14. Timeline (9 weeks)

| Week | Work | Deliverable |
|---|---|---|
| 1 | Requirements, literature survey, data collection | SRS, source list |
| 2 | Knowledge base build, ASR baseline | Vector store, WER report |
| 3 | Symptom extraction and follow-up logic | Extractor with schema |
| 4 | RAG pipeline and retrieval evaluation | Retrieval metrics |
| 5 | Rule engine and urgency classification | Rule YAML, test suite |
| 6 | TTS and full voice pipeline | End-to-end CLI demo |
| 7 | UI and deployment | Working web app |
| 8 | Evaluation, ablation, expert review, user test | Evaluation report |
| 9 | Report, slides, demo video | Final submission |

## 15. Risks and mitigations

| Risk | Mitigation |
|---|---|
| Weak Rajasthani ASR | Hindi first; Rajasthani is a stretch goal |
| Hallucinated medical advice | Answer only from retrieved text; fallback; safety merge; post-check |
| ASR errors change meaning | Transcript confirmation step |

| Risk | Mitigation |
| --- | --- |
| Messy guideline PDFs | Start with a few high-quality documents; manual cleaning |
| Free GPU limits | Use faster-whisper int8 on CPU and API LLM for the demo |
| Liability and misuse | Disclaimers, no diagnosis, documented scope |
| Time overrun | Cut stretch goals; keep the safety layer and evaluation |

# 16. Documents you need to prepare

| # | Document | Purpose |
|---|---|---|
| 1 | Project synopsis / proposal (2 to 3 pages) | Approval from faculty |
| 2 | PRD (this document) | Defines what is built and why |
| 3 | SRS (Software Requirements Specification) | Formal functional and non-functional requirements |
| 4 | Literature survey | Existing symptom checkers, Indic ASR, medical RAG papers |
| 5 | System design document | Architecture, flow diagrams, API and DB design |
| 6 | Dataset and knowledge-base document | Sources, licences, cleaning steps, chunking choices |
| 7 | Safety and ethics note | Scope, disclaimers, red-flag rule sources |
| 8 | Test plan and test cases | The 100 triage cases and software tests |
| 9 | Evaluation report | Metrics, ablation, expert review, user test |
| 10 | User manual | How to use the app |
| 11 | Final project report | Full write-up in college format |
| 12 | PPT and demo video | Presentation and backup demo |
| 13 | GitHub repository with README | Code, setup steps, results |
| 14 | Weekly progress log | Evidence of continuous work |

## 17. Suggested repository structure

```
health-triage/
  README.md
  docker-compose.yml
  backend/
    app/ (main.py, api/, services/asr.py, extractor.py, rules.py, rag.py,
generator.py, tts.py)
    rules/redflags.yaml
    tests/
  frontend/ (React app)
  knowledge_base/ (raw_pdfs/, processed/, build_index.py)
  evaluation/ (test_cases.csv, run_eval.py, results/)
  docs/ (PRD, SRS, design, report)
```

## 18. Likely viva questions and answers

- **How do you know the advice is good?** It is validated against WHO IMCI and NHM danger-sign protocols, with expert review of sample outputs and an ablation showing each safety layer's effect. We do not claim clinical validation.
- **What if the LLM hallucinates?** It may only use retrieved text, falls back when retrieval is weak, and a rule engine and post-check can override it.
- **What is the most dangerous error?** Under-triage, calling an emergency harmless. We measure it separately and target zero on the protocol set.
- **Why not just use ChatGPT?** It is not grounded in Indian guidelines, has no deterministic safety layer, and is not voice-first in Hindi.
- **What would you do next?** Dialect support, offline mode, a pilot with ASHA workers, and clinical review.

## 19. Success criteria

- End-to-end Hindi voice demo works reliably.
- Zero under-triage on the protocol test set.
- Ablation shows measurable safety gains from RAG and rules.
- All required documents and the repository are complete and reproducible.